

Under-Constrained Circom Circuit, Security Review

Target: `Square.circom`, a Circom circuit proving knowledge of a square root, with a Groth16 backend

Type: Demonstration review on intentionally vulnerable code **Method:** snarkjs proof-of-concept, vulnerable branch (master) and remediated branch (fixed)

This report documents one finding. A passing proof-of-concept reproduces the exploit on `master`; the same scenario, run against the remediated circuit on `fixed`, shows the attack neutralised.

H-01, Under-constrained signal breaks soundness (High)

The circuit is meant to prove knowledge of a private `root` such that `root^2 == n`, where `n` is public. It does not. A signal is assigned without being constrained, so a prover can produce a verifying proof using a `root` that is not a square root of `n`. In the proof-of-concept the public input is `n = 9` and the forged proof uses `root = 7`, even though `7^2 = 49`. The verifier accepts it.

Root cause

Circom distinguishes assignment from constraint:

- `<--` assigns a value to a signal in the witness. It adds nothing to the R1CS.
- `<==` assigns and adds the corresponding constraint.
- `===` adds a constraint without assigning.

The circuit uses `<--` where it needed `<==`:

```
template Square() {
  signal input root;    // private
  signal input n;       // public
  signal sq;
  sq <-- root * root;   // assigns sq, but does NOT constrain sq == root*root
  sq === n;
}
```

The only constraint emitted is `sq === n`. The relation `sq == root * root` is never added, so `root` appears in no constraint at all. Compiled with `--00`, the R1CS has a single constraint and `root` is free; under the default optimizer the circuit reduces to zero constraints, which is the same defect seen at its limit. Either way, the prover is free to choose `root`.

Attack

The witness vector is `[1, n, root, sq]`. An honest prover for `n = 9` supplies `root = 3`, the witness calculator sets `sq = 9`, and `sq === n` holds. To forge:

1. Generate the honest witness, then overwrite only the `root` slot with `7`. Leave `sq = 9`.
2. The forged witness `[1, 9, 7, 9]` still satisfies the sole constraint `sq === n` (`9 == 9`).

3. Run `groth16.prove` on the forged witness. `snarkjs` trusts the witness and produces a proof.
4. The verifier checks the proof against the public input `n = 9` and accepts it.

The proof now attests "I know a square root of 9" while the prover used 7. Used as an authorization gate, this is a full bypass.

Proof of concept

`test/exploit.cjs`, run on `master`. It builds the forged witness by serializing a `.wtns` file directly, then proves and verifies:

```
public n                : 9
forged root (7^2=49)    : 7
honest proof verifies   : true
forged proof verifies   : true
```

The honest proof verifies (the circuit works for a real prover), and the forged proof also verifies (soundness is broken).

Recommendation

Bind `sq` with `<==` so the constraint `sq == root * root` is emitted:

```
sq <== root * root;
sq === n;
```

The R1CS then enforces `root * root == n`. On the `fixed` branch the same proof-of-concept reports `forged proof verifies : false`: the forged witness violates `sq == root * root` ($9 \neq 49$), so its proof fails verification while the honest proof still passes. As a general rule, every signal a circuit's guarantee depends on must be pinned by a constraint (`<==` or an explicit `===`). A `<--` assignment with no accompanying constraint is the canonical under-constrained-circuit bug and should be flagged on sight.

Severity

High. A soundness break lets a prover produce a verifying proof for a statement they cannot satisfy. In a circuit acting as an authorization or state-transition gate, the property the proof was meant to guarantee is fully bypassed.

Scope and disclaimer

`Square.circom` is intentionally vulnerable code written to demonstrate audit methodology end to end. It is not production code and must never be deployed. The finding above is a real vulnerability in this demo circuit, reproduced with an executable proof-of-concept, not an invented severity.